Making the Teamscale API Available in Claude Code

Three MCP Server Implementations Compared

Marcel Bruckner

🔌 Why MCP?

The **Model Context Protocol** connects AI assistants to external tools and data.

An MCP server exposes:

- **Tools** — actions Claude can invoke (e.g. "list projects", "get findings")
- **Resources** — data Claude can read (read-only GET endpoints)
- **Resource Templates** — parameterized resources (e.g. `/projects/{project}/findings`)

Claude Code has native MCP support via its **plugin system**.

I built three plugins that bridge Claude Code to the Teamscale REST API.

🗺️ The Three Approaches

	Python OpenAPI	Python Custom	TypeScript Custom
Strategy	Auto-generate from OpenAPI spec	Hand-craft curated tools	Hand-craft curated tools
API coverage	Full (121 endpoints)	13 selected tools	13 selected tools
Server code	65 lines	439 lines	535 lines
Runtime	Python + uv	Python + uv	Node.js 18+
MCP SDK	FastMCP 3.x	mcp[cli] (official)	@modelcontextprotocol/sdk

1 Python OpenAPI

Auto-generated from the Teamscale OpenAPI spec

Python OpenAPI — How It Works

```
Teamscale Instance                          FastMCP OpenAPIProvider
        |                                          |
        |  GET /openapi.json                       |
        |------------------------------------------>|
        |                                          |
        |  JSON spec (all endpoints)               |
        |<------------------------------------------|
        |                                          |
        |     Automatically registers:             |
        |     - GET -> Resources                   |
        |     - GET {param} -> Templates           |
        |     - POST/PUT/DELETE -> Tools           |
        |       (if ENABLE_TOOLS is set)           |
```

65 lines of code. No hand-written tool definitions at all.

Python OpenAPI — What Gets Exposed

Type	Count	Examples
Resources	11	`getAllProjects`, `getHealthStatus`, `getAllUsers`
Resource Templates	46	`getFindings`, `getBranches`, `getCode`
Tools (opt-in)	64	`createProject`, `flagFindings`, `uploadReport`
Total	**121**	

All Teamscale REST endpoints, mapped automatically.

Python OpenAPI — Trade-offs

✅ Advantages

- Near-zero maintenance
- New Teamscale endpoints appear automatically
- Full API surface available
- Only 65 lines of code

⚠️ Disadvantages

- 121 endpoints flood Claude's tool list
- No higher-level business logic
- Claude often picks wrong endpoint or passes wrong parameters
- Env vars **required** at startup
- Slower startup (HTTP fetch of spec)

2 Python Custom

Hand-crafted tools for Teamscale workflows

Python Custom — How It Works

```
teamscale-openapi.json
        |  openapi-python-client (code generation)
        v
teamscale-rest-api-client/       Generated, type-safe Python client
        |  imported by
        v
server.py                        13 hand-written tools with business logic
        |  registered on
        v
FastMCP Server (stdio)
```

Each tool accepts optional `server`, `user`, `access_key` params with env-var fallback.

Key UX advantage: no setup wizard needed — Claude asks for credentials when needed.

🔑 Zero-Config Connection Model

```
list_projects(server=?, user=?, access_key=?)
                    |
         +----------+----------+
         |                     |
 Param provided?        Env var set?
   server="..."      TEAMSCALE_URL="..."
         |                     |
         v                     v
    Use param             Use env var
         |                     |
         +------> Client <-----+
```

- 👋 **First time users:** just install the plugin, Claude asks for details
- ⚡ **Power users:** set env vars once, never prompted again
- 🔀 **Multi-instance:** pass different servers per tool call, even in one session

Python Custom — The 13 Tools

Project & Config

Tool	Description
`list_projects`	List all projects
`get_project_id`	Find project by repo URL
`get_project_connectors`	Debug connectors
`get_project_branches`	List live branches
`delete_teamscale_project`	Delete project

Worker Logs

Tool	Description
`get_worker_logs`	All log entries
`get_worker_log_warnings`	Warnings only

Findings

Tool	Description
`get_findings_list`	Fetch all findings
`get_findings_count_per_file`	Spot outlier files
`get_findings_count_per_check`	Spot noisy checks

Verification

Tool	Description
`verify_project_dashboards`	Check dashboards
`verify_architecture`	Detect violations

Python Custom — Key Pattern: `@teamscale_tool`

```python
@MCP.tool()
@teamscale_tool
async def list_projects(
    server: str | None = None,
    user: str | None = None,
    access_key: str | None = None,
    fetch = None,  # injected by decorator
) -> list[dict]:
    """List all projects on a Teamscale instance."""
    client = resolve_connection(server, user, access_key)
    response = await fetch(
        get_all_projects.asyncio_detailed(client=client)
    )
    return [p.to_dict() for p in response.parsed]
```

The decorator injects a `fetch` helper that handles HTTP errors uniformly, then **hides itself** from the tool's public signature.

Python Custom — Trade-offs

✅ Advantages

- Curated tools with clear intent
- Higher-level logic (aggregation, verification)
- Uniform error handling via decorator
- **Zero-config install** — env vars optional
- Claude picks the right tool more reliably

⚠️ Limitations

- Each new endpoint requires manual work
- Only 13 of 121 endpoints covered

> 💡 Python 3.14 is what the prototype uses — minimum version can be lowered.

3 TypeScript Custom

Same curated tools, different runtime

TypeScript Custom — How It Works

```
teamscale-openapi.json
        |  @hey-api/openapi-ts (code generation)
        v
teamscale-rest-api-client/       Generated TypeScript client
        |  imported by
        v
server.ts                        13 hand-written tools (same as Python)
        |  tsc compile
        v
dist/server.js  -->  McpServer (stdio)
```

`start-server.js` bootstraps everything: npm install, client generation, tsc, then runs the server.

Client can be **pre-generated and committed** instead — see upcoming slides.

TypeScript Custom — Same Tools, Different SDK

```
server.tool(
  "list_projects",
  "List all projects on a Teamscale instance.",
  { ...connectionParams },
  async ({ server: srv, user, access_key }) => {
    const client = resolveConnection(srv, user, access_key);
    const projects = await teamscaleFetch(
      await getAllProjects({ client })
    );
    return textResult(projects);
  }
);
```

Uses Zod schemas for parameter validation. `teamscaleFetch` provides the same error handling as Python's `@teamscale_tool`.

TypeScript Custom — Trade-offs

✅ Advantages

- **Node.js 18+** widely available
- Official MCP TypeScript SDK (Tier 1, most mature)
- Identical tool set to Python custom
- Zod validation on all parameters
- **Zero-config install** — env vars optional

⚠️ Limitations

- More verbose (535 vs 439 lines)
- Each new endpoint requires manual work

📦 Client Generation: Two Strategies

Current: Generate at Startup

```
User installs plugin
        |
  start-server.{py,js}
        |
  generate-client.sh   <-- runs on first start
        |
  server starts
```

Simple, always matches the bundled spec. Adds a few seconds to first launch.

Future Option: Pre-Generate & Commit

```
Developer runs generate-client.sh
        |
  commits generated client to repo
        |
  plugin ships with pre-built client
        |
  server starts immediately
```

Faster startup. Pin client to a specific Teamscale version. Review generated code in PRs.

📦 Why Pre-Generating Matters

	Generate at Startup	Pre-Generate & Commit
Startup speed	Slower (first run)	⚡ Instant
Version control	Implicit (bundled spec)	🔒 Explicit, pinned
Reviewability	Generated code not in git	✅ Visible in PRs
Teamscale version coupling	Tied to bundled spec	Controlled upgrades
Developer effort	None	Run script when upgrading

This is a natural next step: the generation scripts already exist. Just commit the output instead of gitignoring it.

📊 Side-by-Side Comparison

Dimension	Python OpenAPI	Python Custom	TS Custom
Endpoints	121	13	13
Code	65 lines	439 lines	535 lines
Runtime	Python + uv	Python + uv	Node.js 18+
Zero-config install	❌ Env vars required	✅	✅
Auto new endpoints	✅	❌ Manual	❌ Manual
Business logic	❌	✅	✅
Error handling	Generic HTTP	🛡️ Decorator	🛡️ Helper fn
SDK maturity	Tier 1	Tier 1	Tier 1

🚀 Installation — Three Steps

1. Register the marketplace:

```
/plugins marketplace add https://github.com/MarcelBruckner/teamscale-claude-marketplace.git
```

2. Install the plugin you want:

```
/plugins install teamscale-typescript-custom
```

3. Reload:

```
/reload-plugins
```

That's it. No env vars needed — Claude asks for credentials on first use.

🧭 Which Approach Fits?

"We want full coverage with minimal effort"

→ Python OpenAPI — but Claude struggles with 121 undifferentiated tools

"We want Claude to be effective at specific workflows"

→ Either Custom plugin — 13 curated tools with clear intent

"We want the best of both worlds"

→ **Hybrid**: OpenAPI for broad read-only access + Custom for curated workflows. They can coexist as separate plugins.

🎯 Recommendation

Use a **Custom plugin** as the primary interface for developers.

- ✅ Curated tools give Claude better context and more reliable results
- ✅ **Zero-config install** — developers can start immediately
- ✅ Pre-generate the client for version control and fast startup
- ✅ Choose **TypeScript** or **Python** based on your team's runtime

Consider adding the **OpenAPI plugin** alongside for exploratory/advanced use cases.

Questions?